

Deep Learning Fundamentals Final Solution

Module 1: Basics

1. [Mid 1] What happens if the learning rate is too large during training. [1 Mark]

If the learning rate is too large, the model may **overshoot the minimum and fail to converge (oscillate or diverge)**.

2. [Mid 1] What is the main purpose of stochastic gradient descent (SGD)? [1 Mark]

The main purpose of stochastic gradient descent (SGD) is to **efficiently optimize the loss function by updating parameters using small random batches of data**.

3. [Mid 1] Which problem is caused by high bias: underfitting or overfitting? [1 Mark]

High bias causes **underfitting**.

4. [Mid 1] Which type of model capacity usually leads to underfitting? [1 Mark]

Low-capacity models usually lead to underfitting.

5. [Mid 1] Name one common regularization technique used in deep learning. [1 Mark]

One common regularization technique is **dropout**.

6. [Mid 1] Which model has better generalization: low bias-high variance or high bias-low variance? [1 Mark]

A **low bias–high variance** model generally has better generalization than high bias–low variance.

7. [Mid 1] Derive a general formula to compute the number of binary input patterns for n inputs. [1 Mark]

For each input, there are 2 possible values (0 or 1). Since the inputs are independent, the total number of possible combinations is obtained by multiplying choices for each input.

So, for n inputs:

Number of binary input patterns = 2^n

8. [Mid 1] Compute the mean and variance for the dataset: {1, 3, 5, 7, 9}. [1 Mark]

$$\text{Mean } (\bar{x}) = (1 + 3 + 5 + 7 + 9) / 5 = 25 / 5 = 5$$

Variance (σ^2):

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

Where,

- x_i = Data values
- $\bar{x}$ = Mean
- N = Total number of data points

$$\sigma^2 = [(1 - 5)^2 + (3 - 5)^2 + (5 - 5)^2 + (7 - 5)^2 + (9 - 5)^2] / 5$$

$$\sigma^2 = (16 + 4 + 0 + 4 + 16) / 5 = 40 / 5$$

$$\sigma^2 = 8$$

9. [Mid 1] A model shows training accuracy = 98% and validation accuracy = 70%. Analyze whether the model is overfitting or underfitting and justify numerically. [1 Mark]

The model is **overfitting**. It performs very well on the training data with an accuracy of 98%, which means it makes very few mistakes during training. However, its validation accuracy is only 70%, so it makes many more mistakes on new, unseen test data. The large difference of 28% between training and validation accuracy shows that the model has learned the training data too well but does not generalize properly.

10. Justify the need for regularization techniques in deep feedforward networks.

- **High Model Complexity:** Deep networks have a large number of parameters, causing them to more likely memorize training data instead of learning general patterns.
- **Prevention of Overfitting:** Regularization discourages overly complex models, improving performance on test data.
- **Improved Generalization:** By constraining weights, regularization helps the network learn robust and meaningful features rather than noise.
- **Control of Large Weights:** Techniques like L1 and L2 regularization punish large weights, stabilizing training and reducing sensitivity to input variations.
- **Limited Training Data:** When data is insufficient compared to model size, regularization becomes essential to avoid poor generalization.

11. Explain the role of linear algebra concepts (vectors, matrices, tensors) in deep learning models.

1. **Vectors:** Inputs such as images, text, or audio are represented as vectors. Each neuron has a weight vector and a bias, and the outputs (activations) are also vectors passed between layers.
2. **Matrices:** A fully connected layer computes weighted sums of inputs, using matrix multiplication, and then applies a nonlinear activation function:

$$y = f(Wx + b)$$

Where,

- **W:** Weight matrix
- **x:** Input of the layer
- **y:** Output of the layer
- **b:** Bias

Inputs are processed in batches as matrices for faster computation, and weight updates in backpropagation are efficiently handled using matrix operations.

3. **Tensors:** Tensors extend vectors (1D) and matrices (2D) to higher dimensions, allowing representation of complex data like images (3D) and videos (4D/5D). All data, parameters, and operations in deep learning frameworks are handled as tensors, enabling parallel computation and GPU acceleration.

12. Differentiate between low-capacity and high-capacity models with respect to underfitting and overfitting.

Low-Capacity Models:

- Have **limited flexibility** and **fewer parameters**.
- Tend to **underfit** because they cannot capture complex patterns in the data.
- Show **high bias and low variance**, performing poorly on both training and test data.

High-Capacity Models:

- Are **highly flexible** with **many parameters**.
- Tend to **overfit** by learning noise along with the true pattern.
- Show **low bias and high variance**, performing very well on training data but poorly on unseen data.

13. Describe the bias–variance trade-off with a suitable example.

Bias: The error due to overly simplistic assumptions in the model.

- **High-bias models** are too simple to capture the true pattern in the data.
- They tend to **underfit**, performing poorly on both training and test data.

Variance: The error due to extra sensitivity to the training data.

- **High-variance models** are too complex and fit noise along with the real pattern.
- They tend to **overfit**, performing very well on training data but poorly on test data.

The Bias-Variance Trade-off:

- Increasing model complexity reduces bias but increases variance.
- Decreasing model complexity reduces variance but increases bias.
- The goal is to find a balance where total error (bias + variance) is minimized.

Example: Suppose you are predicting exam scores based on study hours,

- **High Bias:** A constant model that predicts the same score for everyone. It ignores study time and underfits.
- **High Variance:** A very complex model that memorizes each student's score. It fits training data perfectly but fails on new students.
- **Balanced Model:** A simple but flexible regression that captures the general trend without memorizing noise.

14. Explain how stochastic gradient descent helps in optimizing deep neural networks.

In **Stochastic Gradient Descent (SGD)**, the gradient is computed using one training example, and parameters are updated immediately:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta; x_i, y_i)$$

Where,

- **θ :** Model parameters
- **η :** Learning rate
- **L :** Loss for a single example (x_i, y_i)

Why SGD Helps in Deep Neural Networks:

- **Computational Efficiency:** Updates parameters using **one sample or a small minibatch**, making **training faster and scalable** to large datasets.
- **Better Generalization:** The noise in updates prevents overfitting and helps the model find flatter, more generalizable minima.
- **Escaping Saddle Points and Shallow Minima:** Randomness in updates helps move through flat regions and avoid getting stuck in poor solutions.
- **Frequent Updates:** More frequent parameter updates allow faster initial learning and smoother convergence.
- **Practical Flexibility:** Supports online and minibatch learning, which is well-suited for deep neural networks and modern hardware.

15. Given $y = wx + b$, $x = 4$, $w = 2$, $b = 1$, learning rate $\eta = 0.1$, compute the updated value of w after one gradient descent step using squared error loss.

Given:

- **Model:** $y = wx + b$
- **Initial Weight:** $w = 2$
- **Input:** $x = 4$
- **Bias:** $b = 1$
- **Learning Rate:** $\eta = 0.1$
- **Loss:** squared error, $L = (y - \hat{y})^2$

Step 1: Gradient Prediction

$$\hat{y} = wx + b = 2(4) + 1 = 9$$

Step 2: Gradient of Loss (L) With Respect to Weight (w)

$$\frac{\partial L}{\partial w} = 2(\hat{y} - y)x = 2(9 - y)(4) = 8(9 - y)$$

Step 3: Gradient Descent Update

$$w_{\text{new}} = w - \eta \frac{\partial L}{\partial w}$$

$$w_{\text{new}} = 2 - 0.1(8(9 - y)) = 2 - 0.8(9 - y)$$

$$w_{\text{new}} = 2 - 7.2 + 0.8y = 0.8y - 5.2$$

Thus, the updated value of w , for some given target output y , is $0.8y - 5.2$.

16. Two models have the following errors:

Model A $\rightarrow$ Bias = low, Variance = high

Model B $\rightarrow$ Bias = high, Variance = low

Analyze which model will generalize better.

Model B will generalize better. Model A has low bias and high variance, which means it fits the training data very closely but is **sensitive to noise and changes in the data**. As a result, it is likely to **overfit**. Model B has high bias and low variance, which means it makes **simpler assumptions** and is **more stable** across different datasets. Although it **may underfit slightly**, its **predictions are more consistent**.

17. Given two learning rates (0.001 and 0.5), evaluate which one is more suitable based on convergence behavior and loss oscillations.

A learning rate of 0.001 is more suitable. With 0.5, the learning rate is **too large**.

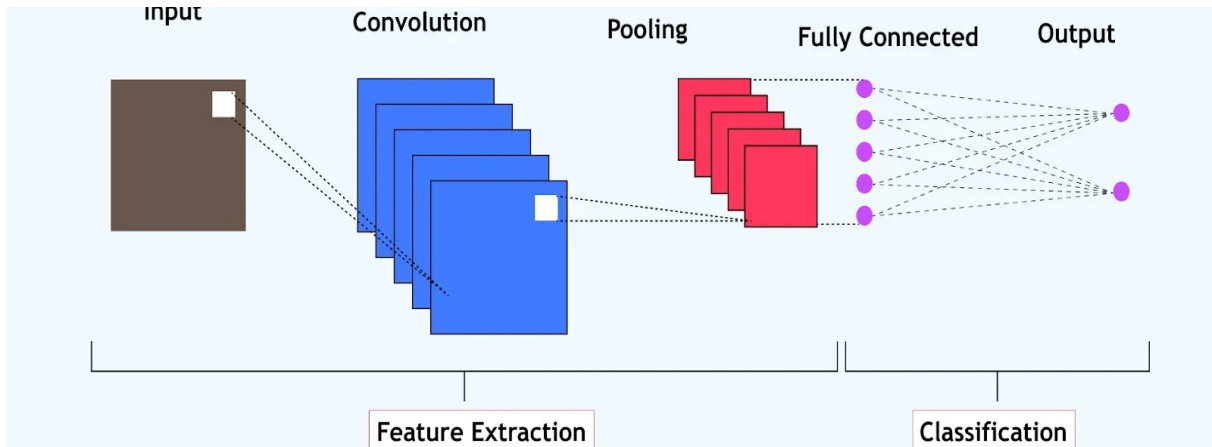
Updates are **aggressive**, which often causes the **loss to oscillate**, **overshoot** the minimum, or even **diverge** instead of converging. With 0.001, the learning rate is **small and stable**. Although convergence is **slower**, the **loss decreases smoothly and steadily** toward the minimum.

Module 2: CNN

1. [Mid 1] Compare Activation Function and Loss Function. [1 Mark]

An **activation function** determines a neuron's output by applying a nonlinear transformation to its weighted input, while a **loss function** measures the error between predicted and actual outputs to guide model training.

2. [Mid 1] Draw the Architecture of CNN. Explain the role of pooling layers in CNN architectures. [5 Marks]



Role of Pooling Layer in CNNs:

- Reduces spatial size of feature maps (downsampling).
- Lowers computational cost and memory usage.
- Retains important features using operations like max or average pooling.
- Provides translation invariance to small input changes.
- Helps reduce overfitting by decreasing number of parameters.
- Allows the network to focus on the most significant features.

3. [Mid 1] What is Feature map in CNN? Analyze how stride and padding affect feature map dimensions. **OR** Analyze how stride and padding affect feature map dimensions. [5 Marks]

A **feature map** in a CNN is the output of a convolution layer obtained after applying a filter to the input, representing the presence of specific features (like edges or textures) in different spatial locations.

Stride is the number of pixels the filter moves at each step.

- **Small stride (e.g. 1):** The filter moves one pixel at a time, producing a larger feature map with more detailed spatial information.
- **Large stride (e.g. 2 or more):** The filter skips pixels, producing a smaller feature map and reducing spatial resolution.

Padding adds extra pixels (usually zeros) around the input.

- **No padding (valid convolution):** The filter only slides where it fully fits inside the input, causing the feature map size to shrink.
- **With padding (same convolution):** Padding allows the filter to cover border pixels, helping preserve the input size in the output feature map.

The output size of a feature map is given by:

$$\frac{N - F + 2P}{S} + 1$$

Where,

- **N** = Input Size
- **F** = Filter Size
- **P** = Padding
- **S** = Stride

4. [Mid 1] Calculate the output size of a convolution layer for input size 32×32 , filter size 3×3 , stride = 1, padding = 0. [5 Marks]

Output size of a convolutional layer is found using the following formula:

$$\text{Output Size} = \frac{\text{Input Width} - \text{Kernel Size} + 2(\text{Padding})}{\text{Stride}} + 1$$

Given:

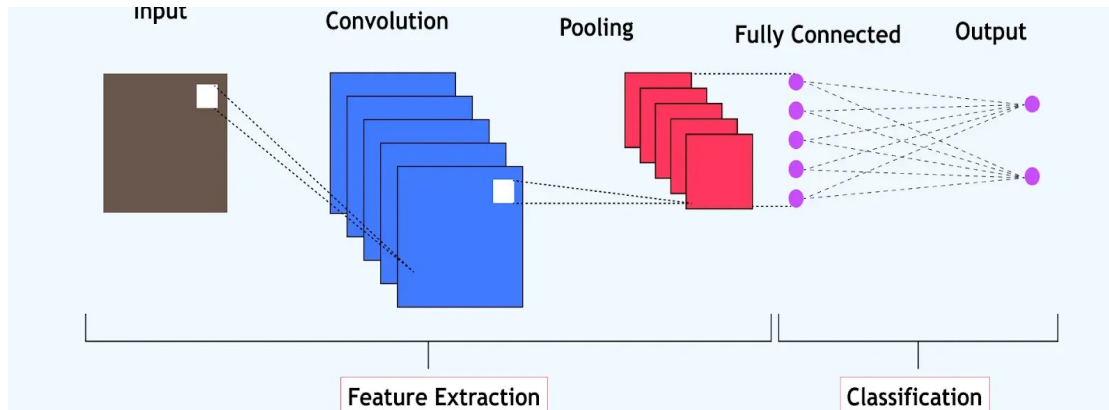
- **Input Width** = 32
- **Kernel / Filter Size** = 3
- **Stride** = 1
- **Padding** = 0

$$\text{Output Size} = \frac{32 - 3 + 0}{1} + 1 = 29 + 1 = 30$$

Thus, the output size of the convolutional layer is 30×30 .

5. Explain the convolution operation in CNNs with respect to feature extraction.

In **Convolutional Neural Networks (CNNs)**, the convolution operation is the key mechanism used for feature extraction from input data such as images.



How convolution works

- A small matrix called a filter or kernel slides over the input image.
- At each position, element-wise multiplication is performed between the filter and the corresponding input region, followed by summation.
- This produces a value in a new matrix called a feature map.

Mathematically, convolution applies the same filter across the entire input, which allows the network to detect patterns regardless of their location.

Role in feature extraction

- **Early layers:** Filters learn to detect simple features such as edges, corners, and textures.
- **Deeper layers:** Feature maps capture more complex patterns like shapes, object parts, or textures formed by combining earlier features.
- Each filter specializes in detecting a specific feature, and multiple filters extract multiple features simultaneously.

6. Given accuracy values before and after pooling (92% → 88%), evaluate whether pooling is beneficial and justify.

In this case, pooling slightly reduces the model's accuracy from 92% to 88%, showing a decrease of 4%. This drop indicates that **some detailed spatial information useful for prediction is lost during pooling**. However, **pooling also reduces model complexity and computational cost and helps prevent overfitting** by making the network more robust to small variations in the input. Therefore, **even though accuracy decreases slightly, pooling can still be beneficial overall if the goal is to improve efficiency and generalization** rather than achieving the highest possible accuracy.

7. Describe sparse interactions and parameter sharing in CNNs.

- **Sparse interactions:** Each neuron connects only to a small local region (receptive field) instead of the entire input, reducing parameters and focusing on local features.
 - **Parameter sharing:** The same filter (weights) is applied across the input, producing feature maps and greatly reducing parameters while detecting features regardless of position.
8. Compute the number of parameters in a CNN layer with 32 filters of size 3×3 and input depth = 3.

Given:

- **Number of Filters** = 32
- **Filter Size** = 3×3 [filter height $\times$ filter width]
- **Input Depth** = 3

Step 1: Calculate number of parameters per filter

Parameters per filter = (filter height $\times$ filter width $\times$ input depth) + 1

Parameters per filter = $(3 \times 3 \times 3) + 1 = 27 + 1 = 28$

Step 2: Calculate total number of parameters

Total number of parameters = Parameters per filter $\times$ Number of filters

Total number of parameters = $28 \times 32 = 896$

So, the CNN layer has 896 trainable parameters.

9. Analyze how increasing stride from 1 to 2 changes the output size for a 28×28 input.

For an input size of 28×28 , assuming filter size of 3×3 and padding of 0,

If Stride = 1,

$$\text{Output Size} = \frac{28 - 3 + 0}{1} + 1 = 25 + 1 = 26$$

Thus, output size would be 26×26

If Stride = 2,

$$\text{Output Size} = \frac{28 - 3 + 0}{2} + 1 = 12.5 + 1 = 13.5 \approx 13$$

Thus, output size would be 13×13

Thus,

- With **stride 1**, the filter moves one pixel at a time, producing a larger output with more spatial detail.
- With **stride 2**, the filter moves two pixels at a time, producing a smaller output and reducing spatial resolution.

10. Evaluate the impact of different activation functions on CNN performance.

ReLU (Rectified Linear Unit):

- Most widely used in CNNs
- Avoids vanishing gradients, enabling faster training.
- Produces sparse activations and improves convergence.
- Generally gives best overall performance.

Sigmoid:

- Outputs values between 0 and 1, useful for probabilities.
- Suffers from vanishing gradient problem.
- Leads to slow and inefficient training in deep CNNs.

Tanh:

- Outputs values between -1 and 1 and is zero-centered.
- Performs better than sigmoid.
- Still affected by vanishing gradients, limiting deep network performance.

Leaky ReLU:

- Allows small gradient for negative inputs.
- Reduces dead neuron problem of ReLU.
- Provides more stable training and can slightly improve performance.

11. Given a 5×5 input and a 3×3 kernel, compute the convolution output.

Given:

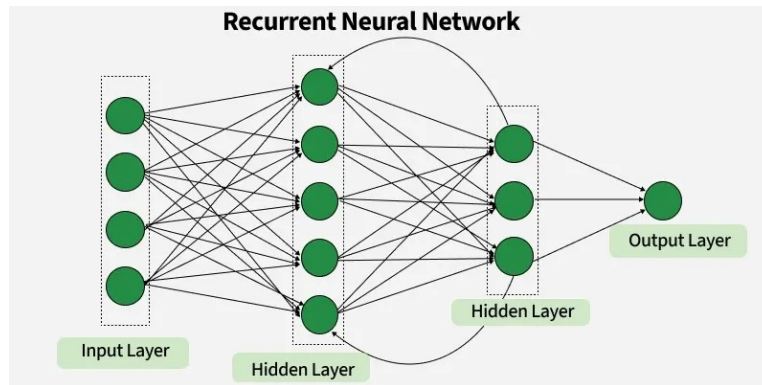
- **Input Width = 5**
- **Kernel Size = 3**
- **Assuming stride of 1 and padding of 0**

$$\text{Output Size} = \frac{5 - 3 + 0}{1} + 1 = 2 + 1 = 3$$

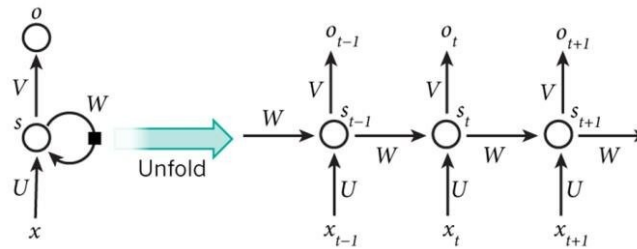
Thus, the output size of the convolutional layer is 3×3 .

Module 3: RNN

1. [Mid 1] Draw the architecture of RNN. What is unfolding in recurrent neural networks.
OR Explain the concept of unfolding in recurrent neural networks. [4 Marks]



In **recurrent neural networks (RNNs)**, unfolding (also called unrolling) is a way to visualize and understand how an RNN processes sequential data over time.



An RNN uses the same set of weights at every time step and maintains a hidden state that carries information from previous inputs. Unfolding represents this process by expanding the RNN across multiple time steps, where each time step looks like a separate layer but all layers share the same parameters. This makes it easier to see how an input sequence is processed step by step and how information flows from earlier time steps to later ones.

Unfolding is especially important for training RNNs because it allows the use of **Backpropagation Through Time (BPTT)**. By unrolling the network, errors can be propagated backward across time steps so that the shared weights are updated based on their influence on the entire sequence. Basically, unfolding converts a loop-based RNN into a chain-like structure over time, helping us analyze sequential behavior and train the network effectively.

2. [Mid 1] Describe the vanishing gradient problem in RNNs. [3 Marks]

The **vanishing gradient problem** in RNNs occurs when **gradients become extremely small during training**, making it difficult to learn long-term dependencies. During Backpropagation Through Time (BPTT), gradients are repeatedly multiplied across many time steps, and if these values are less than 1, they shrink exponentially as they propagate backward. As a result, earlier time steps receive almost no weight updates, causing the network to focus mainly on recent inputs and fail to capture long-term relationships in the data.

3. **[Mid 1]** Evaluate the effectiveness of LSTM networks over vanilla RNNs. **[3 Marks]**

LSTM (Long Short-Term Memory) networks have the following improvements compared to vanilla (regular) RNN networks:

- They overcome the vanishing gradient problem faced by vanilla RNNs.
- LSTMs include a memory cell that helps store information over long time intervals.
- They have input, forget, and output gates control how information is stored, updated, and passed forward.
- LSTMs preserve important information while forgetting irrelevant data.
- LSTMs can capture long-range dependencies that simple RNNs cannot.
- LSTM training is more stable compared to vanilla RNNs.
- LSTMs perform better on tasks like language modeling, speech recognition, and machine translation.

4. **[Mid 1]** Consider a simple RNN with one hidden unit and one output unit. The update rules are:

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

$$y = W_{hy} \cdot h_t + b_y$$

Parameters given:

Initial hidden state $h_0 = 0$

Input Sequence [1, 0, 1]

Weights: $W_{xh} = 1$, $W_{hh} = 0.5$, $W_{hy} = 2$

Biases $b_h = 0$, $b_y = 0$

Compute hidden state (h_1, h_2, h_3)

Compute Outputs(y_1, y_2, y_3) **[4 Marks]**

Ans:

Hidden state calculations

- **Time step 1 ($x_1 = 1$):** $h_1 = \tanh(1 \cdot 1 + 0.5 \cdot 0) = \tanh(1) = 0.76$
- **Time step 2 ($x_2 = 0$):** $h_2 = \tanh(1 \cdot 0 + 0.5 \cdot 0.76) = \tanh(0.38) = 0.36$
- **Time step 3 ($x_3 = 1$):** $h_3 = \tanh(1 \cdot 1 + 0.5 \cdot 0.36) = \tanh(1.18) = 0.83$

Output calculations

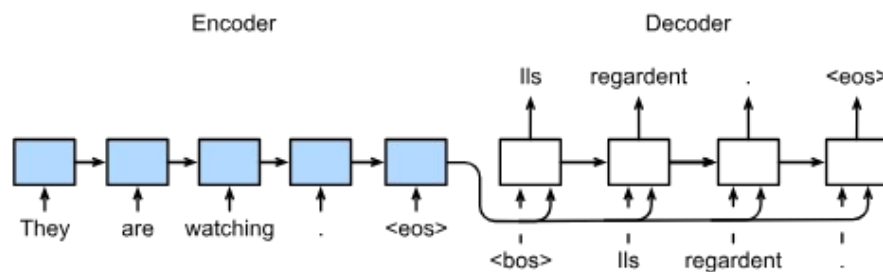
- **Output at time step 1:** $y_1 = 2 \cdot 0.76 = 1.52$
- **Output at time step 2:** $y_2 = 2 \cdot 0.36 = 0.72$
- **Output at time step 3:** $y_3 = 2 \cdot 0.83 = 1.66$

5. [Mid 1] Analyze the role of recurrent connections in speech recognition tasks. [3 Marks]

- **Modeling temporal dependencies:** Recurrent connections allow the model to retain information from previous time steps, helping interpret sounds based on earlier inputs.
- **Context awareness:** Recurrent connections allow the model to use past context to correctly recognize phonemes influenced by neighboring sounds.
- **Handling variable-length inputs:** Recurrent connections help to process speech sequences of different lengths without fixed input size.
- **Capturing continuity:** Recurrent connections help to maintain smooth representation of continuous speech signals over time.

6. Explain the working of sequence-to-sequence models.

Sequence-to-sequence (seq2seq) models are neural network architectures designed to **transform an input sequence into an output sequence**, even when the two sequences have different lengths.



A seq2seq model consists of two main parts:

- **Encoder**
 - Reads the input sequence one element at a time.
 - Compresses the input into a context vector (hidden state).
 - The context vector captures the overall meaning of the input sequence.
- **Decoder**
 - Uses the context vector produced by the encoder.
 - Generates the output sequence step by step.
 - Produces one output token at each time step.
 - Each output depends on the previous outputs and the encoder's context.

7. How can a simple RNN be applied for character-level text generation?

A simple RNN generates character-level text by learning to predict the next character from previous ones. Text is split into characters and encoded (e.g., one-hot). The RNN processes one character at a time, updating its hidden state as memory. During training, it takes a character at time t and predicts the character at $t + 1$, learning patterns in the text. For generation, a seed character is given, the RNN predicts the next character, and this output is fed back repeatedly to produce a sequence.

8. Analyze the limitations of simple RNNs for long-term dependencies.

- **Vanishing gradients:** Gradients shrink during back propagation through time (BPTT), limiting learning from early time steps.
- **Information loss:** Earlier inputs fade, so the model focuses on recent data.
- **Limited memory:** A single hidden state cannot store long or complex sequences effectively.
- **Unstable training:** Repeated multiplications make training slow and unstable.
- **Poor long-range modeling:** Struggle to capture dependencies across distant time steps.

9. Evaluate the performance of RNNs compared to traditional Auto-Regressive (AR) models for time-series forecasting. Discuss strengths and limitations of each approach.

RNNs	Auto-Regressive (AR) Models
Model complex and non-linear temporal patterns.	Model linear relationships between values.
Learn dependencies directly from data.	Depend on past values of the series.
Handle multivariate time series.	Mostly suited for univariate time series.
Adapt to changing patterns over time.	Limited adaptability to change.
Computationally expensive.	Computationally efficient.
Require large amounts of data.	Work well with smaller datasets.
Struggle with long-term dependencies without LSTM/GRU.	Poor at capturing long-term dependencies.
Harder to interpret.	Highly interpretable.
Few explicit assumptions.	Require assumptions like stationarity.

10. Why RNN shares the same weights across all time steps?

- Using the same weights ensures the network applies the same transformation at every time step, allowing it to recognize patterns regardless of when they occur in the sequence.
- Weight sharing greatly reduces the number of parameters, making the model simpler, faster to train, and less prone to overfitting.
- Because the same weights are reused, an RNN trained on shorter sequences can also process longer or shorter sequences without changing its structure.
- Shared weights allow the hidden state to carry information forward in a consistent manner, enabling the network to learn temporal relationships.

Module 4: Model Evaluation

1. [Mid 2] Two classification models are evaluated on a dataset with 1000 samples, where 950 are negative and 50 are positive.

Model A Confusion Matrix			Model B Confusion Matrix		
	Predicted Positive	Predicted Negative		Predicted Positive	Predicted Negative
Actual Positive	30	20	Actual Positive	40	10
Actual Negative	120	830	Actual Negative	200	750

Calculate Accuracy for both models.

Calculate Precision, Recall, and F1-score for the positive class.

Identify which model performs better for the imbalanced dataset. [5 Marks]

For Model A:

- **Accuracy** = $\frac{30+830}{1000} = \frac{860}{1000} = 0.86$
- **Precision** = $\frac{30}{30+120} = \frac{30}{150} = 0.20$
- **Recall** = $\frac{30}{30+20} = \frac{30}{50} = 0.60$
- **F1-Score** = $\frac{2 \times 0.20 \times 0.60}{0.20 + 0.60} = \frac{0.24}{0.80} = 0.30$

For Model B:

- **Accuracy** = $\frac{40+750}{1000} = \frac{790}{1000} = 0.79$
- **Precision** = $\frac{40}{40+200} = \frac{40}{240} = 0.167$
- **Recall** = $\frac{40}{40+10} = \frac{40}{50} = 0.8$
- **F1-Score** = $\frac{2 \times 0.167 \times 0.80}{0.167 + 0.80} = \frac{0.267}{0.967} = 0.276$

Thus, for an imbalanced dataset, **Model B is better** because it has significantly higher **recall (0.80)** for the positive class, meaning it detects more actual positive cases, which is more important than accuracy in such scenarios.

2. [Mid 2] A dataset contains 1000 samples, where 950 belong to class 0 and 50 belong to class 1. A classifier predicts all samples as class 0. Calculate the accuracy of the model. Calculate the recall for the positive class. Explain whether accuracy is a reliable metric for this dataset. [5 Marks]

Total samples = 1000

Correct predictions = 950 (all class 0 correctly predicted)

$$\text{Accuracy} = \frac{950}{1000} = 0.95 = 95\%$$

Recall (for positive class = class 1):

True Positives (TP) = 0 (no class 1 predicted correctly)

False Negatives (FN) = 50

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{0}{0 + 50} = 0$$

Although accuracy is high (95%), the model completely fails to identify the positive class (recall = 0), so it is **not a good model for imbalanced data**.

3. [Mid 2] Compare computational cost of grid search with 5 hyperparameters vs random search with 10 trials. [5 Marks]

Grid Search

- If each of the 5 hyperparameters has k possible values, **total evaluations** = k^5
- **Example:** If each hyperparameter has 5 values, there will be $5^5 = 3125$ model trainings.

Random Search

- For a random search with 10 trials, **total evaluations** = 10

Thus, grid search has **exponentially higher computational cost as hyperparameters increase**, while random search keeps the cost fixed. Therefore, **random search is much more computationally efficient for large hyperparameter spaces**.

4. [Mid 2] Explain the importance of model evaluation in deep learning. [3 Marks]

1. **Measuring Performance:** Helps determine how well a model performs its task using metrics such as accuracy, precision, recall, F1-score, and loss values.
2. **Detecting Overfitting and Underfitting:** By comparing training and test performance, model evaluation identifies whether the model is overfitting or underfitting.
3. **Model Selection:** Enables objective comparison of different models or configurations to select the best-performing one.
4. **Hyperparameter Tuning:** Guides the adjustment of parameters like learning rate, batch size, and network depth to improve model performance and generalization.

5. [Mid 2] Evaluate when random search is preferred over grid search. [3 Marks]

1. **Large Hyperparameter Space:** When there are many hyperparameters or many possible values, grid search becomes computationally expensive, while random search samples fewer combinations.
2. **Limited Computational Resources:** Random search requires less time and computing power because it does not test every possible combination.
3. **Continuous Parameter Ranges:** For parameters like learning rate that have continuous values, random search can sample directly from the range.
4. **Unequal Parameter Importance:** If only a few hyperparameters strongly affect performance, random search is more likely to explore useful values.

6. [Mid 2] Compute RMSE for actual values {2, 4, 6} and predicted values {3, 5, 5}. [3 Marks]

Step 1: Calculating Errors

- $(3 - 2) = 1, (5 - 4) = 1, (5 - 6) = -1$

Step 2: Calculating Squared Errors

- $1^2 = 1, 1^2 = 1, -1^2 = 1$

Step 3: Calculating Mean Squared Error (MSE)

$$MSE = \frac{1+1+1}{3} = \frac{3}{3} = 1$$

Step 4: Calculating Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE} = \sqrt{1} = 1$$

Thus the RMSE for the given values is 1.

7. Describe commonly used performance metrics for classification problems.

1. **Accuracy:** Measures the proportion of correctly predicted instances out of the total instances.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

It is useful when the dataset is balanced, but it may be misleading for imbalanced datasets.

2. **Precision:** Measures the proportion of correctly predicted positive instances among all instances predicted as positive.

$$\text{Precision} = \frac{TP}{TP+FP}$$

High precision means the model produces fewer false positives.

3. **Recall (Sensitivity):** Measures the proportion of actual positive instances that are correctly identified by the model.

$$\text{Recall} = \frac{TP}{TP+FN}$$

High recall indicates that the model detects most of the positive cases.

4. **F1-Score:** The harmonic mean of precision and recall. It provides a balance between the two metrics.

$$\text{F1} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

It is particularly useful when there is an uneven class distribution.

5. **Specificity (True Negative Rate):** Measures the proportion of actual negative cases that are correctly classified.

$$\text{Specificity} = \frac{TN}{TN+FP}$$

It indicates how well the model avoids false alarms.

8. Explain the purpose of baseline models.

A **baseline model** is a simple model used as a **reference point to evaluate more complex machine learning or deep learning models**. Baseline models help **measure whether new models actually improve performance**, simplify early development, and detect potential issues in data or model design. They also **guide further experimentation and optimization** by providing a standard for comparison.

9. Analyze the advantages and limitations of grid search.

Advantages of Grid Search

1. **Systematic Exploration:** Grid search evaluates every combination of specified hyperparameters. This ensures a thorough and structured search of the parameter space.
2. **Easy to Implement and Understand:** It is simple to use and widely supported in machine learning libraries. The concept is straightforward because it tests all combinations in a predefined grid.
3. **Reproducible Results:** Since the search space is predefined and deterministic, grid search produces consistent and reproducible results across runs.
4. **Suitable for Small Parameter Spaces:** When the number of hyperparameters and possible values is small, grid search can effectively identify the optimal configuration.

Limitations of Grid Search

1. **High Computational Cost:** Grid search becomes very expensive when the number of hyperparameters or their possible values increases.
2. **Inefficient Search:** Many combinations tested may be unnecessary or irrelevant, which wastes computation time.
3. **Limited Coverage of Parameter Space:** Grid search only evaluates values specified in the grid. If the optimal value lies between two grid points, it will not be discovered.
4. **Not Scalable for Deep Learning Models:** Deep learning models often have many hyperparameters and require long training times, making grid search impractical for large-scale problems.

10. Given TP = 50, FP = 10, FN = 20, TN = 120, compute accuracy, precision, and recall.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} = \frac{50+120}{50+120+10+20} = \frac{170}{200}$$

$$\text{Accuracy} = 0.85 = 85\%$$

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{50}{50+10} = \frac{50}{60}$$

$$\text{Precision} = 0.8333 = 83\%$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{50}{50+20} = \frac{50}{70}$$

$$\text{Recall} = 0.7143 = 71.43\%$$

11. Analyze two models using confusion matrices and identify the better one for an imbalanced dataset.

For **imbalanced datasets**, accuracy alone is not reliable because the majority class can dominate the result. Instead, metrics such as precision, recall, and F1-score should be analyzed from the confusion matrix.

Steps to Compare Two Models:

1. **Examine the Confusion Matrices:** Each confusion matrix provides **TP** (True Positives), **FP** (False Positives), **FN** (False Negatives), and **TN** (True Negatives)
2. **Calculate key metrics such as precision, recall, and F1 score.**
3. **Focus on Recall and F1-score:** For imbalanced datasets, correctly identifying the minority class is important. **A model with higher recall and F1-score for the minority class is generally preferred.**

Decision Rule:

- If Model A has higher accuracy but low recall for the minority class, it may be misleading.
- If Model B has higher recall and F1-score, it is usually the better model for imbalanced data.

12. Evaluate whether accuracy is a reliable metric for a dataset with 95% negative samples.

Accuracy is not a reliable metric for a dataset with 95% negative samples because the data is highly imbalanced. A model could simply predict all samples as negative and still achieve about 95% accuracy while completely failing to detect positive cases. This makes accuracy misleading since it does not reflect performance on the minority class. For such datasets, metrics like precision, recall, and F1-score are more appropriate because they better evaluate how well the model identifies the minority class.

Module 5: Auto-encoders and Generative Models

1. [Mid 2] Explain the concept of undercomplete autoencoders. **OR** What is an undercomplete autoencoder? [1 Mark]

An **undercomplete autoencoder** is an autoencoder where the hidden (latent) layer has fewer neurons than the input layer, forcing the model to learn a compressed representation of the data.

2. [Mid 2] What is the usefulness of autoencoders in representation learning. [1 Mark]

Autoencoders are useful in representation learning because they learn compact, meaningful latent features from raw data automatically without requiring labeled data.

3. [Mid 2] Describe regularized autoencoders. **OR** What are regularized Autoencoders? [1 Mark]

Regularized autoencoders are autoencoders that use constraints or penalties during training to prevent simple copying of input, forcing the model to learn meaningful and robust features.

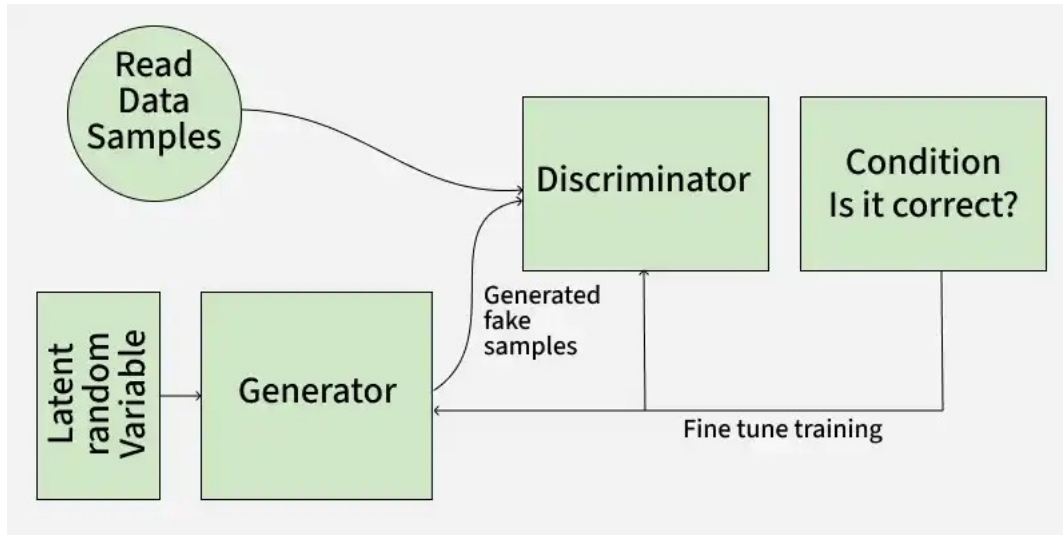
4. [Mid 2] Given reconstruction errors across epochs, determine whether learning is improving. **OR** What does reconstruction error determines across epochs? [1 Mark]

Learning is **improving** if the **reconstruction error decreases over epochs**.

5. [Mid 2] What is reconstruction loss? [1 Mark]

Reconstruction loss is the measure of the difference between the original input and the output reconstructed by the autoencoder, indicating how well the model has learned to represent the data.

6. [Mid 2] Draw the architecture of GAN. Explain its working. [4 Marks]



A Generative Adversarial Network (GAN) consists of two main components:

- **Generator:** Takes a latent random vector as input and generates fake samples.
- **Discriminator:** Takes both real data samples and generated (fake) samples and classifies them as real or fake.

Working:

- A random noise vector (latent variable) is fed into the Generator, which produces fake data samples.
- These generated samples, along with real data samples, are given to the Discriminator.
- The Discriminator evaluates whether the input is real or fake.
- If the discriminator correctly identifies samples, it is reinforced; otherwise, the generator is updated to produce more realistic samples.
- This process continues in a minimax game, where the generator tries to fool the discriminator, and the discriminator tries to correctly distinguish real vs fake.
- Over time, the generator improves and produces highly realistic data.

7. **[Mid 2]** What is Compression Ratio in Autoencoders? An autoencoder has input size 512 and bottleneck size 64. Compute the compression ratio. **[4 Marks]**

Compression ratio in autoencoders is the ratio of the original input size to the size of the bottleneck (latent) representation. It indicates how much the data has been **compressed** into a lower-dimensional representation.

Compression ratio is defined as:

$$\text{Compression Ratio} = \frac{\text{Input Size}}{\text{Latent (Bottleneck) Size}}$$

Given,

- Input size = 512
- Bottleneck size = 64

$$\text{Compression Ratio} = 512 / 64 = 8$$

Thus, the compression ratio = 8:1, meaning the representation is compressed 8 times smaller than the input.

8. Explain how variational autoencoders generate new samples.

Variational Autoencoders (VAEs) generate new samples by learning a probability distribution of the training data rather than a fixed encoding.

During training, **the encoder maps each input into parameters of a probability distribution in the latent space.** Instead of producing a single latent vector, **the model samples a point from this distribution.**

The decoder then takes this sampled latent vector and reconstructs it into data similar to the original input. After training, **new samples can be generated by randomly sampling points from the latent distribution and passing them through the decoder,** which produces new data resembling the training dataset.

9. Analyze the training challenges of GANs.

The challenges of Generative Adversarial Networks (GANs) include:

- **Training Instability:** The generator and discriminator are trained simultaneously in a competitive process, which can cause oscillations and difficulty in reaching convergence.
- **Mode Collapse:** The generator may produce only a small variety of outputs, failing to capture the full diversity of the data distribution.
- **Vanishing Gradients:** If the discriminator becomes too strong, the generator receives very small gradients and cannot learn effectively.
- **Hyperparameter Sensitivity:** GAN training is sensitive to choices like learning rate, batch size, and network architecture.
- **Balance Between Networks:** The generator and discriminator must learn at similar rates; if one becomes much stronger than the other, training becomes ineffective.

10. Evaluate the usefulness of autoencoders in representation learning.

Autoencoders are useful in representation learning because they **automatically learn compact latent features from raw data without requiring labeled datasets or manual feature engineering**. By compressing high dimensional data into a lower dimensional representation, **they capture important patterns and structures in the data**, making them **effective for tasks like dimensionality reduction and feature extraction**. However, without proper constraints or regularization, **autoencoders may simply learn to copy the input rather than meaningful representations**.

11. Compare reconstruction loss values of undercomplete and regularized autoencoders.

Undercomplete autoencoders typically aim to minimize reconstruction loss by compressing the input into a smaller latent space, so their **reconstruction loss is often relatively low** if the bottleneck still captures essential information. However, the compression constraint may sometimes slightly increase the loss compared to unconstrained models.

Regularized autoencoders include additional constraints such as sparsity penalties or noise. Because of these constraints, they **often show higher reconstruction loss during training**, but they learn more robust and meaningful representations and generalize better to new data.

In comparison, undercomplete autoencoders usually achieve lower reconstruction loss, while regularized autoencoders may have slightly higher loss but better feature learning and generalization.

12. Analyze generator and discriminator loss curves to identify GAN instability.

By examining the generator and discriminator loss curves during training, we can identify the GAN instability with the following cases:

- **Stable training:** Both generator and discriminator losses fluctuate but remain within a reasonable range. Neither network consistently dominates the other.
- **Discriminator loss near zero:** Indicates the discriminator is too strong, easily distinguishing real and fake samples, which can cause vanishing gradients for the generator.
- **Generator loss increasing rapidly:** Suggests the generator is failing to improve and cannot fool the discriminator.
- **Large oscillations in both losses:** Indicates unstable training where the networks keep overpowering each other without converging.
- **Sudden drop in generator diversity with low loss:** May indicate mode collapse, where the generator produces limited outputs.

13. Given KL divergence and reconstruction error values, evaluate the performance of a VAE.

To evaluate a Variational Autoencoder (VAE), both reconstruction error and KL divergence are considered because the VAE loss combines these two terms.

- **Low reconstruction error** indicates that the **decoder can accurately reconstruct the input data from the latent representation.**
- **Low or moderate KL divergence** shows that the **learned latent distribution is close to the desired prior distribution** (usually a standard normal distribution).

Evaluation:

- **Low reconstruction error + reasonable KL divergence:** Good performance. The VAE reconstructs data well while maintaining a well-structured latent space.
- **Low reconstruction error + very high KL divergence:** The model reconstructs well but the latent space may not follow the desired distribution, reducing generative quality.
- **High reconstruction error + low KL divergence:** The latent space is well regularized, but the model is not reconstructing data effectively.
- **High values for both:** Poor model performance.